

CLI Commands

lambda	# Start REPL
// REPL Commands: quit, help, clear	
lambda script.ls	# Eval functional script
lambda run script.ls	# Run procedural script
lambda --help	# Show help

Validation:

lambda validate file.json -s schema.ls	# With schema
lambda validate file.json	# Default schema

Type System

Scalar Types:

null bool int float decimal	Primitives
string symbol binary datetime	Primitives
path	
i8 i16 i32 i64	Sized signed integers
u8 u16 u32 u64	Sized unsigned integers
f16 f32 f64	Sized floats

Container Types:

range, 1 to 10	Range (inclusive both ends; also “a” to “z”)
array, [123, true]	Array of values
map, {key: 'symbol'}	Map
element, <div class: bold, "text"> >	Element

Type Operators:

int string	Union type
int & number	Intersection
int?	Optional (int null)
int*	Zero or more
int+	One or more
int[]	Array of ints (same as int*)
int[5]	Array of exactly 5 ints
[int*]	Bracket form: array of 0+ ints
[int+]	Bracket form: array of 1+ ints
fn (a: int, b: string) bool	Function type
fn int	Same as fn () int
{a: int, b: bool}	Map type
<div id:symbol; >	Element type

Type Declarations:

type User = {name: string, age: int}	Map type alias
type Point = (float, float)	Tuple type
type Result = int error	Union type

Object Types

Definition:

type Point { x: float, y: float }	// Fields only
type Counter {	
value: int = 0,	// Default value
fn double() => value * 2	// Functional
fn add(n: int) => value + n	
pn inc() { value = value + 1 }	// Procedural
}	
type Circle : Shape { radius: float }	// Inheritance

Lambda Script Cheatsheet

Literals & Access:

let p = <Point x: 1.0, y: 2.0>	Object literal
let c = <Counter>	All defaults
p.x	Field access
c.double()	Method call
<Point *:p, x: 5.0>	Copy and override

Type Checking (nominal only):

p is Point	true
p is object	true
p is map	true (objects are map-compatible)
{x: 1} is Point	false (plain maps don't match)

Constraints:

type User {	
name: string that (len(-) > 1),	// Field constraint
age: int that (0 <= ~ <= 150),	
that (~.name != "admin")	// Object constraint
}	

Self reference ~:

type Vec {	
x: float, y: float,	
fn len() => math.sqrt(x**2 + y**2)	
fn scale(f) => <Vec *:~, x: ~.x*f, y: ~.y*f>	// ~ = self
}	

Literals

Numbers:

42	Integer
42n	Arbitrary precision integer
3.14	Float
1.5e-10	Scientific notation
123.45m	Decimal
1e-3m	Decimal (negative exponent)
inf nan	Special values
0xFF	Hexadecimal int literal

Sized numeric literals (postfix suffix):

42i8 -128i8 1000i16 100000i32	Signed
100i64	
255u8 60000u16 3000000000u32	Unsigned
1000u64	
0.5f16 3.14f32 2.7f64	Float

Strings & Symbols:

"hello"	String
"multi-line string"	Multi-line string — a literal newline may appear inside the quotes
'symbol'	Symbol
symbol	Unquoted symbol
'name' == "name"	false — symbol is not string
"" == null	false — empty string is a string value
not ""	true — empty string is falsy
'' is invalid: empty symbols do not exist.	

Binary:

b'\xDEADBEEF'	Hex binary
b'\64QUVGRw=='	Base64 binary

DateTime:

t'2025-01-01T14:30:00Z'	DateTime
t'2025-04-26' is date	Sub-type: date
t'10:30:00' is time	Sub-type: time

Member properties:

dt.date dt.year dt.month dt.day	Date parts
dt.time dt.hour dt.minute	Time parts
dt.second dt.millisecond	
dt.weekday dt.yearday dt.week	Derived
dt.quarter	
dt.unix dt.timezone	Zone / epoch
dt.utc_offset dt.utc dt.local	

Formatting:

dt.format("YYYY-MM-DD")	Pattern
dt.format('iso')	Named format

Constructors:

datetime() today() justnow()	Current date/time
datetime(2025, 4, 26, 10, 30)	From parts
date(2025, 4, 26)	Date / time only
time(10, 30, 45)	

Collections:

[1, 2, 3]	Array
{a: 1, b: 2}	Map
<div id: "main">	Element

Indexing & Slicing:

arr[0]	First element
arr.0	Alt. syntax for const index
arr[1 to 3]	Slice (indices 1, 2, 3)
map.key	Map field access
map["key"]	Map field by string
"hello"[1 to 3]	“ell” — string slicing
'hello'[1 to 3]	‘ell’ — symbol slicing
"café"[2 to 3]	“fé” — UTF-8 aware

Namespaces (via import with bare URI):

import svg: 'http://www.w3.org/2000/svg'	
import xlink: 'http://www.w3.org/1999/xlink'	
<svg.rect svg.width: 100>	Namespaced tag & attribute — desugars to
	<svg.rect svg: {width: 100}>
elem.svg.width	Chained sub-map access
elem.svg	{width: 100} sub-map
svg.rect	Qualified symbol
symbol("href", 'xlink_url')	Namespaced symbol

Variables & Declarations

Let Expressions (immutable):

(let x = 5, x + 1, x * 2)	Single binding
(let a = 1, b = 2, a + b)	Multiple bindings

Let Statements (immutable):

let x = 42	Immutable binding
let y : int = 100	With type annotation
let a: i8 = 42i8	Sized type annotation
let b: f32 = 3.14f32	Sized float annotation
let a = 1, b = 2	Multiple bindings
x = 10	ERROR E211: cannot reassign let binding

Var Statements (mutable, `pn` only):

<code>var x = 0</code>	Mutable variable
<code>var y: int = 42</code>	With type annotation
<code>var a: int[] = [1, 2, 3]</code>	Array of ints
<code>var b: float[] = [0.1, 0.2]</code>	Array of floats
<code>x = x + 1</code>	OK: reassignment
<code>x = "hello"</code>	OK: type widening (int → string)
<code>y = "oops"</code>	ERROR E201: annotated type enforced

Operators

Arithmetic:	
Operator	Meaning
<code>+</code>	addition
<code>-</code>	subtraction
<code>*</code>	multiplication
<code>/</code>	division
<code>div</code>	integer division
<code>%</code>	modulo
<code>**</code>	exponentiation

Spread *:		
Given	Expression	Result
<code>let a = [1, 2, 3]</code>	<code>(*a, *[10, 20])</code>	<code>(1, 2, 3, 10, 20)</code>

Comparison:	
Operator	Meaning
<code>==</code>	equal
<code>!=</code>	not equal
<code><</code>	less than
<code><=</code>	less or equal
<code>></code>	greater than
<code>>=</code>	greater or equal
<code>==</code> performs structural deep equality on all types:	
<code>[1, 2] == [1, 2]</code>	true (array)
<code>{a:1, b:2} == {b:2, a:1}</code>	true (map, order-independent)
<code>[1] == [1.0]</code>	true (numeric promotion)
<code>(1 to 3) == [1, 2, 3]</code>	true (cross-type sequence)

Logical:	
Operator	Meaning
<code>and</code>	logical and
<code>or</code>	logical or
<code>not</code>	logical not

Type & Set:	
Operator	Meaning
<code>is</code>	type check
<code>in</code>	membership
<code>to</code>	range
<code> </code>	union
<code>&</code>	intersection
<code>!</code>	exclusion

Query: type-based search	
<code>? .?</code>	recursive descendant search
<code>expr[T]</code>	child-level query (direct only)

Vector Arithmetic:	
Expression	Result
<code>1 + [2, 3]</code>	<code>[3, 4]</code> scalar broadcast
<code>[1, 2] * 2</code>	<code>[2, 4]</code> scalar broadcast
<code>[1, 2] + [3, 4]</code>	<code>[4, 6]</code> element-wise

Use `++` for list/array concat: `[1,2] ++ [3,4] = [1,2,3,4]`.

Pipe Expressions

Pipe > with current item ~:	
<code>[1,2,3] > ~ * 2</code>	<code>[2,4,6]</code> - map over items
<code>[1,2,3] > sum</code>	<code>6</code> - aggregate (no ~)
<code>users > ~.age</code>	<code>[12,20,62]</code> - extract field
<code>['a','b'] > {i::~#, v::~}</code>	<code>~#</code> = index/key

Filter with that:	
<code>[1,2,3,4,5] that (~ > 3)</code>	<code>[4,5]</code>
<code>users that (age >= 18) > ~.</code>	filter then map
<code>name</code>	
<code>[1,2,3] > ~ ** 2 that (~ > 5)</code>	<code>13 (4+9)</code>
<code> > sum</code>	

Spreading in Array Literals: pipe and filter results flatten	
<code>[1, [2,3] > ~, 4, 5]</code>	<code>[1, 2, 3, 4, 5]</code>
<code>[0, [1,2,3] > ~ * 10, 99]</code>	<code>[0, 10, 20, 30, 99]</code>
<code>[1, [3,5,7] that (~ > 4), 9]</code>	<code>[1, 5, 7, 9]</code>

Query Expressions

Results in document order (depth-first, pre-order).

Query ? — attributes + all descendants:	
<code>html?</code>	all <code></code> at any depth
<code>html?<div class: string></code>	<code><div></code> with class attr
<code>data?int</code>	all int values in tree
<code>data?(int string)</code>	all int or string values
<code>data?{name: string}</code>	maps with string 'name'
<code>data?{status: "ok"}</code>	maps where status == "ok"

Self-inclusive query .? — self + attributes + all descendants:	
<code>div.?<div></code>	includes div itself if it matches
<code>el.?int</code>	self + all int values in subtree
<code>42.?int</code>	<code>[42]</code> — trivial self-match

Child-level query [r] — direct attributes + children only (no recursion):	
<code>[1, "a", 3, true][int]</code>	<code>[1, 3]</code> — int items
<code>{name: "Alice", age: 30}[string]</code>	<code>["Alice"]</code>
<code>]</code>	
<code>el[element]</code>	direct child elements only
<code>el[string]</code>	attr values + text children

File Output (procedural only)

The target can be a string, a symbol, or a path.		
<code>output(data, 'output.txt')</code>	File under CWD	
<code>output(data, "./temp/output.txt")</code>	Relative path	
<code>"")</code>		
<code>output(data, "output.txt", {mode: "append"})</code>	Append mode	
<code>output(data > format('json'), "output.json")</code>	Format then write	

Data type determines output format:

- String: raw text (no formatting)
- Binary: raw binary data
- Other types: Lambda/Mark format

Control Flow

If Expressions (parenthesized condition, else required):

```
if (x > 0) "positive" else "non-positive"  Simple
if (score >= 90) "A" else if (score >= 80) "B" else "C"  Chained
if (x > 0) "pos" else { "neg" }  Block else
```

If Statements (block body, else optional):

```
if x > 0 { "positive" }  No else
if condition { something() }  Both blocks
else { otherThing() }
if x > 0 { compute() } else "default"  Expression else
```

Both forms share the same else syntax: `else expr`,

`else { stam },` OR `else if ....`

Match Expressions:

```
// Type patterns or Literal
match value {
  case 200: "OK"           // case literal
  case string: "text"      // case type
  case int | float: "number" // case type union
  case Circle:             // case object type
    3.14 * ~.r ** 2
  default: "other"
}

// String pattern arms (full-match semantics)
type digits = \(d+)
type alpha = \(a+)
match input {
  case digits: "number"      // case named pattern
  case alpha: "word"
  default: "other"
}
```

For Expressions: (produce spreadable arrays — pipe/filter also spread, see above)

Expression	Result
<code>for (x in [1, 2, 3]) x * 2</code>	<code>[2, 4, 6]</code>
<code>for (i in 1 to 5) i * i</code>	<code>[1, 4, 9, 16, 25]</code>
<code>for (x in data) if (x > 0) x else 0</code>	conditional body
<code>[for (i in 1 to 2) for (j in 1 to 2) i * j]</code>	<code>[1, 2, 2, 4]</code> nested for-expressions flatten
<code>[for (x in []) x]</code>	<code>[]</code> empty for produces spreadable null, which is skipped
<code>for (k at {a: 1, b: 2}) k</code>	<code>['a', 'b']</code> iterate a map by key
<code>for (k, v at {a: 1, b: 2}) k ++ v</code>	<code>['a1', 'b2']</code> key and value — see note

Known defect. The paired `at` form binds **both** names to the key, so `for (k, v at {a: 1, b: 2}) k ++ v` currently yields

[`'aa'`, `'bb'`], not [`'a1'`, `'b2'`]. The bare `for k, v in c` form binds key and value correctly. Tracked as LR02-8.

```
For Expression Clauses: let, where, group by, order by,
limit, offset
for (x in data where x > 0) x          // filter
for (x in data, let sq = x*x) sq       // let binding
for (x in [3,1,2] order by x) x        // (1,2,3)
for (x in [3,1,2] order by x desc) x   // (3,2,1)
for (x in data limit 5 offset 10) x    // pagination
for (x in data, let y=x*2
  where y>5 order by y desc limit 3) y
// group by → g is an <group> element (keys=attrs, members=
  children)
for (x in sales group by x.region into g)
  {region: g.region, n: len(g)}
```

```
Joins (on): relate comma sources; ? = left join
for (o in orders, c in customers on o.cust_id == c.id)
  {id: o.id, name: c.name}           // inner equi-join
for (o in orders, c? in customers on o.cust_id == c.id)
  {id: o.id, name: c}               // left join (c=null on
  miss)
for (o in os, c in cs on o.a==c.a and o.b==c.b) {...} // multi
-key
```

```
For Statements:
for item in collection { transform(item) }
```

Procedural Control (in `pn`):

<code>var x = 0</code>	Mutable variable
<code>while (c) { break; continue;</code>	Loop and its jumps
<code>return x }</code>	

Assignment Targets (in `pn`):

<code>x = 10</code>	Variable reassignment (var only)
<code>arr[i] = val</code>	Array element reassignment
<code>obj.field = val</code>	Map field reassignment
<code>elem.attr = val</code>	Element attribute reassignment
<code>elem[i] = val</code>	Element child reassignment

Functions

Function Declaration:

<code>fn add(a: int, b: int) int { a</code>	Statement body
<code>+ b }</code>	
<code>fn multiply(x: int, y: int) =></code>	Expression body
<code>x * y</code>	
<code>let square = (x) => x * x</code>	Anonymous function
<code>pn f(n) { var x = 0; while (x <</code>	Procedural function
<code>n) { x = x + 1 }; x }</code>	
<code>pn advance(pos: float[], vel:</code>	Array parameters
<code>float[], n: int) { ... }</code>	

Concurrency (pn only)

```
pn worker() { return receive()^ }

pn main() {
  let h = start(worker)          // scoped child, opaque
  identity handle
```

```
send(h, "job")^                // bounded FIFO mailbox (
  default 1024)
print(wait(h)^)                // wait for T^E result
}

start(worker, [job], {mode: 'task'}) // args array + launch
  options (S13.1.1v2)

wait(h, timeout: 10)           // timeout does not cancel h
select(h1, h2, timeout: 100)^  // first completed handle
sleep(5)^                      // shared libuv timer
self()                         // current handle
cancel(h)                      // idempotent cancellation
  request
io.read("file.txt")^           // async local file read
```

No `async`/`await` keywords are used. Normal block exit joins children; error exit cancels then joins. A `start` operand may not capture an outer `var` by reference—copy to `let` or use messages. Imported JS Promises are wait-able; exported Lambda `pn`s return Promises to JavaScript.

Advanced Features:

<code>fn f(x?:int)</code>	optional param
<code>fn f(x=10)</code>	default param value
<code>fn f(...)</code>	variadic args
<code>f(b:2, a:1)</code>	named param call
<code>fn outer(n) { fn inner(x)=>x+n;</code>	closure
<code>inner }</code>	

String Patterns

Define named patterns for string validation and matching. Uses regex-like syntax integrated into the type system.

Definition:

<code>type digits = \(\d+</code>	one or more digits
<code>type email = \(\w+ "@\w+ ".\." a</code>	email-like
<code>[2,6])</code>	
<code>type ws = \(\s+</code>	whitespace
<code>type keyword = 'if' 'else' </code>	symbol literal union
<code>'for'</code>	

Type check (`is`) — full-match semantics:

<code>"hello@world.com" is email</code>	true
<code>"abc" is email</code>	false
<code>"123" is digits</code>	true

Character classes inside `\(...)`: `a` digit, `w` word, `s` whitespace, `a` alpha, `.` any char, `...` any string
Reserved inside the island only — quote to match one literally: `\("d" w+)`.

Quantifiers: `?` optional, `+` one or more, `*` zero or more, `[n]` exactly `n`, `[n,m]` range

Domain: `\(...)` matches strings, `\symbol(...)` matches symbols; the tag is checked before the content, and pattern bodies always use string literals for content. Given `type SymIdent = \symbol(a w*)`:

Form	Result
<code>'foo' is SymIdent</code>	<code>true</code>
<code>"foo" is SymIdent</code>	<code>false</code> — string value, symbol pattern

Inline (unnamed) patterns work anywhere a type does:

<code>"abc" is \(\a+)</code>	<code>true</code>
<code>fn f(x: \(\d+)) => x</code>	Parameter annotation
<code>match s { case \(\d+): "num"</code>	Match arm
<code>default: "other" }</code>	

System Functions

Type:

`int(v)` `int64(v)` `float(v)` `decimal(v)` `string(v)` `symbol(v)` `binary(v)`
`number(v)` `type(v)` `len(v)`
`type()` returns the canonical numeric name: `type(42i8) → "i8"`, `type(3.14f32) → "f32"`, `type(1.0f64) → "float"`

Math:

`math.pi` `math.e` `abs(x)` `sign(x)` `min(a,b)` `max(a,b)` `round(x)` `floor(x)`
`ceil(x)` `math.trunc(x)` `math.sqrt(x)` `math.cbrt(x)` `math.hypot(x,y)`
`math.pow(b,e)` `math.log(x)` `math.log2(x)` `math.log10(x)`
`math.log1p(x)` `math.exp(x)` `math.exp2(x)` `math.expm1(x)` `math.sin(x)`
`math.cos(x)` `math.tan(x)` `math.asin(x)` `math.acos(x)` `math.atan(x)`
`math.atan2(y,x)` `math.sinh(x)` `math.cosh(x)` `math.tanh(x)`
`math.asinh(x)` `math.acosh(x)` `math.atanh(x)` `math.random(seed) → [value, new_seed]`

Stats:

`sum(v)` `avg(v)` `math.mean(v)` `math.median(v)` `math.variance(v)`
`math.deviation(v)` `math.quantile(v,p)` `math.prod(v)`

Date/Time:

`datetime()` `today()` `now()` `justnow()` `date(dt)` `time(dt)`
Range:
`s` to `e` creates a range from `s` to `e` (inclusive both ends). Bounds are exact integers or single-codepoint strings.

`range(s,e,step)` creates a range with custom step (exclusive end).
`1 to 5` `[1, 2, 3, 4, 5]`
`"a" to "e"` `["a", "b", "c", "d", "e"]` — character range

`range(0, 10, 2)` `[0, 2, 4, 6, 8]`
String:

`replace(str,old,new)` `split(str,sep)` `join(strs,sep)`
`find(str,pattern)` `normalize(str)` `ord(str)` `chr(int)`
All three accept both plain strings and named patterns as the second argument: Given `type digit = \(\d)`,
`type digits = \(\d+)`, `type ws = \(\s+)`:

Form	Result
<code>replace("a1b2c3", digit, "X")</code>	<code>"aXbXcX"</code>
<code>replace("hello world", ws, " ")</code>	<code>"hello world"</code>
<code>replace("abc", "b", "")</code>	<code>"ac"</code>

split(str, pattern_or_string)	
Form	Result
split("a,b,c", ",")	["a", "b", "c"]
split("a1b2c3", digit)	intended ["a", "b", "c", ""] — see note
split("hello world", ws)	intended ["hello", "world"] — see note
split("a1b2c3", digit, true)	intended ["a", "1", "b", "2", "c", "3", ""] — keep delimiters — see note

Known defect. split splits correctly on a plain **string** delimiter, but a **pattern** delimiter is not split on — the matches are stripped and the remainder is returned as a single element: split("a1b2c3", digit) yields ["abc"], and the keep-delimiters form returns ["a1b2c3"] unchanged. replace and find handle patterns correctly. Tracked as LR09-8.

find(str, pattern_or_string) →	[{value, index}, ...]
Form	Result
find("a1b22c333", digits)	[{value:"1", index:1}, {value:"22", index:3}, ...]
find("hello world", "lo")	[{value: "lo", index: 3}]

Collection:

slice(v,i) slice(v,i,j) set(v) all(v) any(v) reverse(v) sort(v) unique(v) take(v,n) drop(v,n) zip(a,b) fill(n,x) range(s,e,step) map(f,v) filter(f,v) reduce(f,v,init)

Vector:

math.dot(a,b) math.norm(v) math.cumsum(v) math.cumprod(v) argmin(v) argmax(v) diff(v)

I/O:

input(file,fmt) format(data,fmt) print(v) output(data,file) fetch(url,opts) cmd(c,args) error(msg) varg()

Input/Output Formats

Supported Input Types: json, xml, yaml, markdown, csv,

html, latex, toml, rtf, css, ini, math, pdf
input("path/file.md", 'markdown') // Input Markdown

Input with Flavors: e.g. math flavors: latex, typst, ascii
input("math.txt", {'type':'math', 'flavor':'ascii'})

Output Formatting: json, yaml, xml, html, markdown

format(data, 'yaml') // Format as YAML

Modules, Imports & Exports

Import Syntax:

import module_name	Built-in module
import .relative_module	Relative to script's dir
import .path.to.module	Nested relative import
import alias: .module	Import with alias

Export Declarations:

```
pub let PI = 3.14159           // Export variable
pub fn square(x) => x * x      // Export function
pub pn log(msg) { print(msg) } // Export procedure
pub type Score = int           // Export type alias
pub type Counter {            // Export object type
  value: int = 0,
  fn double() => value * 2
}
pub let data = input("f") ^ { {} } // Export, handling err
```

Module Usage:

```
// In math_utils.ls:
pub let PI = 3.14159
pub type Vec2 { x: float, y: float, fn len() =>
  math.sqrt(x**2 + y**2) }

// In main.ls:
import .math_utils
let area = PI * r ** 2
let v = <Vec2 x: 3.0, y: 4.0>
v.len() // 5.0
v is Vec2 // true
```

Error Handling

Creating Errors:

error("Message")	Message only
error("load failed", inner_err)	With an inner error
error({code: 304, message: "div by zero"})	Structured payload

Error Return Types (T^E):

fn parse(s: string) int~ {...}	int or any error
fn divide(a, b) int ~ DivErr {...}	specific error
fn load(p) Config ~ ParseErr IOErr {...}	multi errors

raise **error** , or **propagate error with ^**

```
fn compute(x: int) int~ {
  if (b == 0) raise error("div by zero") // raise error
  let a = parse(input)^ // return immediately on error
  let b = divide(a, x)^ // return immediately on error
  a + b
}
fun()^ // propagate error, discard value
```

e ^ { } — handle the error here (- is the error):

```
let result = divide(10, x) ^ {
  print(^.message) // ^ = the current handler error
  0 // handler value, or raise/return
}
result * 2 // result is clean here
```

e or default rescues any falsy value without seeing it;
x is error tests a soft T | error value.

Operator Precedence (High to Low)

- () [] . ? .? - Primary, query
- + not ! - Unary (not: logical NOT, !: type negation)
- ** - Exponentiation

- * / div % - Multiplicative
- + - - Additive
- < <= > >= - Relational
- == != - Equality
- and - Logical AND
- or - Logical OR
- to - Range
- is in - Type operations (is nan for NaN detection)
- | where - Pipe and Filter

Quick Examples

Data Processing:

```
let data = input("sales.json", 'json')
let total = sum(
  (for (sale in data.sales) sale.amount))
let report = {total: total,
  count: len(data.sales)}
format(report, 'json')
```

Function Definition:

```
fn factorial(n: int) int {
  if (n <= 1) 1 else n * factorial(n - 1)
}
```

Element Creation:

```
let article = <article title: "My Article",
  <h1 "Introduction">
  <p "Content goes here.">
>
format(article, 'html')
```

Comprehensions - Complex data processing:

```
(let data = [1, 2, 3, 4, 5],
 let filtered = (for (x in data)
  if (x % 2 == 0) x else 0),
 let doubled = (for (x in filtered) x * 2), doubled)
```